

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY JNANA
SANGAMA, BELAGAVI - 590018**



An Experimental Learning Activity Report on
“Reliable Transport Protocol Framework using C++”

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering
in
Electronics and Communication Engineering
for the Academic Year: 2025-26

Submitted by

Pragna M	1NT23EC103
Navyashree S J	1NT23EC092
Pragna Sridhar	1NT22EC088

Under the Guidance of

Dr.Karunakara rai
Professor
and
Mr.Amruth Ashok Gadag
Assistant Professor

Department of Electronics and Communication Engineering



An Autonomous Institution, Affiliated to VTU, Nitte University Campus, Bengaluru-560064.

Certificate

This is to certify that the Learning Activity work entitled “*Reliable Transport Protocol Framework using C++* ” has been carried out by *Pragna M (INT23EC103)*, *Navyashree S J (INT23EC092)* and *Pragna Sridhar (INT23EC088)* bonafide students of *Nitte Meenakshi Institute of Technology*, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in the **Department of Electronics and Communication Engineering**, under **Visvesvaraya Technological University**, Belagavi, during the academic year **2025–2026**.

The Learning Activity report has been examined and approved as it meets the academic requirements specified under the autonomous scheme of **Nitte Meenakshi Institute of Technology** for the said degree.

Name of Examiners

Signature

1. **Dr. Karunakar Rai**

.....

2. **Mr. Amruth Ashok Gadag**

.....

COMBINED EXPERIENTIAL LEARNING ACTIVITY PROBLEM STATEMENT					
COURSE 1 DETAILS			COURSE 2 DETAILS		
COURSE TITLE	Computer Networks and Applications		COURSE TITLE	Object-Oriented Programming Language using C++	
COURSE CODE	22ECG61		COURSE CODE	22EC64	
SEMESTER	6		SEMESTER	6	
SECTION	B		SECTION	B	
COURSE INSTRUCTOR	Dr. Karunakara Rai		COURSE INSTRUCTOR	Mr.Amruth Ashok Gadag	
STUDENT(S) TEAM MEMBERS	1NT23EC088 1NT23EC092 1NT23EC103		STUDENT(S) TEAM MEMBERS	1NT23EC088 1NT23EC092 1NT23EC103	
COMBINED PROBLEM STATEMENT					
TOPICS COVERED & MAPPING OF COURSE OUTCOMES			TOPICS COVERED & MAPPING OF COURSE OUTCOMES		
LIST OF TOPICS COVERED		COs	LIST OF TOPICS COVERED		COs
		CO1			CO1
		CO2			CO2
		CO3			CO3
		CO4			CO4
		CO5			CO5
RUBRICS FOR COURSE 1			RUBRICS FOR COURSE 2		
ASSESMENT CRITERIA	ELEMENTS ASSESSED	MAX MARKS	ASSESMENT CRITERIA	ELEMENTS ASSESSED	MAX MARKS
CRITERIA 1	Application of OOP Principles	5	CRITERIA 1	Use of Network Concepts	5
CRITERIA 2	Code Modularity and Reusability	5	CRITERIA 2	Application Layer Implementation	5
CRITERIA 3	Operator Overloading & Exception Handling	5	CRITERIA 3	Security Considerations	5
CRITERIA 4	Code Quality and Documentation	5	CRITERIA 4	System Integration and Output	5
TOTAL MARKS		20	TOTAL MARKS		20
COURSE 1 INSTRUCTOR SIGNATURE			COURSE 2 INSTRUCTOR SIGNATURE		

Acknowledgement

The successful execution of Learning Activity has been a significant milestone, and we take this opportunity to express our heartfelt gratitude to all those who have supported and guided us throughout this journey. Whatever we have achieved is the result of their encouragement and help, and we remain deeply thankful to each one of them.

We express our sincere thanks and seek the blessings of **Dr. N. R. Shetty**, Advisor, *Nitte Meenakshi Institute of Technology*, for his vision and emphasis on Learning Activity and constructivist principles that have greatly enriched our academic experience. We are grateful to **Mr. Rohit Punja**, Administrator, *Nitte Education Trust*, and **Dr. Sandeep Shastri**, Vice President, *Bangalore Campus, Nitte University*, for their strategic leadership and continued support in fostering academic excellence.

We extend our special thanks to our beloved Principal, **Dr. H. C. Nagaraj**, for providing us with the necessary resources, facilities, and motivation to carry out our Learning Activity successfully. Our sincere gratitude goes to **Dr. J. Sudheer Reddy**, *Dean – Academics*, and **Dr. Kiran Aithal**, *Dean – Research & Development*, for their guidance, encouragement, and for creating an environment that promotes innovation and academic growth.

We also convey our deep appreciation to **Dr. Parameshachari B D**, *HoD, Department of Electronics and Communication Engineering* for his constant encouragement, valuable guidance, and for fostering a vibrant learning environment. We extend our heartfelt thanks to our Dr. Karunakar Rai and Mr. Amruth Ashok Gadag *Department of Electronics and Communication Engineering* for his/her unwavering support, timely feedback, and insightful mentorship throughout the Learning Activity. We are also indebted to our parents for their unconditional love, support, and encouragement throughout our academic journey at *Nitte Meenakshi Institute of Technology*, Bengaluru. Finally, we would like to express our appreciation to all those **named and unnamed**—who contributed in any way to our learning and success.

Pragna M	(1NT23EC103)
Navyashree S J	(1NT23EC092)
Pragna Sridhar	(1NT23EC088)

Place: Bengaluru

Date:

Abstract

The increasing demand for fast and reliable content delivery over networks has made optimization an essential requirement in modern communication systems. This project presents the design and implementation of a Content Delivery Optimization System aimed at improving the efficiency of data transmission across networks. The system is developed using C++, incorporating a client-server architecture to manage content requests, data transmission, and caching mechanisms.

The primary focus of this work is **server selection based on delay**, where the system intelligently selects the most suitable server for content delivery by considering network delay. This approach helps in reducing response time and improving overall user experience.

To evaluate the performance of the proposed system, simulations are carried out using the NS3 (Network Simulator 3) platform under varying network conditions and traffic scenarios. Optimization techniques such as efficient routing and load handling are applied to enhance system performance. Key performance metrics including latency, throughput, and packet loss are analyzed to measure the effectiveness of the system.

The results demonstrate that the proposed approach significantly improves data delivery speed, reduces transmission delays, and enhances overall network efficiency. This work provides a practical understanding of content distribution and network optimization techniques, making it suitable for real-world applications in modern content delivery networks.

Contents

Acknowledgement	i
Abstract	ii
List of Figures	iv
Chapter 1 Introduction	1
Chapter 2 Literature Survey	2
Chapter 3 Design Approach and Methodology	4
Chapter 4 Implementation Details	10
Chapter 5 Results and Analysis	11
Chapter 6 Conclusion and Future Scope (Optional)	14
Bibliography	15

List of Figures

Fig 3.2 Code for simulation	5-7
Fig 3.3 Code for MATLAB	8-9
Fig 5.2 Graphical results.....	12-13

Chapter 1

Introduction

The rapid growth of internet usage and digital communication technologies has created an increasing demand for fast, secure, and reliable data transmission systems. Modern applications such as video streaming, cloud services, online gaming, IoT communication, and real-time multimedia applications require efficient transport mechanisms to ensure smooth and uninterrupted data delivery. As network traffic continues to grow, traditional communication methods often face challenges such as packet loss, transmission errors, congestion, high latency, and reduced throughput. These issues can significantly affect network performance and degrade the overall user experience.

To address these challenges, reliable transport protocols play a vital role in ensuring accurate and efficient communication between network devices. A reliable transport protocol is responsible for providing error detection, packet recovery, flow control, sequencing, and retransmission mechanisms to guarantee that data reaches the destination correctly and in order. Protocols such as TCP provide reliability, but they may introduce additional overhead and delays under certain network conditions. Therefore, designing an optimized and flexible transport protocol framework has become an important area of research in computer networks.

In this project, a Reliable Transport Protocol Framework is designed and implemented using C++. The framework follows a client-server architecture in which reliable communication is established between sender and receiver systems. The primary objective of the project is to ensure dependable data transmission even in the presence of network errors, delays, or packet loss. The framework incorporates mechanisms such as packet sequencing, acknowledgments, timeout handling, retransmission strategies, and error control techniques to improve communication reliability and efficiency.

The proposed system aims to simulate the core functionalities of a transport layer protocol while providing flexibility for testing and performance evaluation. By implementing the protocol framework in C++, the project achieves efficient memory management, faster execution, and better control over low-level networking operations. The framework can be extended to support different reliability algorithms and congestion-handling techniques based on application requirements.

To evaluate the effectiveness of the proposed framework, simulations and testing are performed under various network conditions and traffic scenarios. Performance metrics such as latency, throughput, packet delivery ratio, retransmission rate, and packet loss are analyzed to measure the reliability and efficiency of the system. The experimental results demonstrate that the proposed framework improves reliable communication, enhances data delivery performance, and provides a strong foundation for developing advanced transport-layer networking applications.

Chapter 2

Literature review

Reliable data communication has become one of the most important requirements in modern computer networks. With the rapid growth of internet-based applications such as cloud computing, multimedia streaming, online gaming, and distributed systems, the need for efficient and dependable transport protocols has increased significantly. Researchers and developers have proposed several transport-layer mechanisms and frameworks to improve data reliability, reduce packet loss, and enhance network performance under different communication environments. One of the earliest and most widely used reliable transport protocols is the Transmission Control Protocol (TCP). TCP provides reliable communication through mechanisms such as acknowledgments, retransmissions, sequencing, flow control, and congestion control. Studies on TCP have shown that it ensures ordered and error-free delivery of packets, making it suitable for applications requiring high reliability. However, researchers have also identified limitations of TCP, including increased overhead, higher latency, and reduced efficiency in high-speed or lossy networks. These drawbacks motivated the development of alternative reliable transport mechanisms.

Several researchers have explored the implementation of reliable transport protocols using simulation and programming frameworks. Protocols based on Stop-and-Wait ARQ, Go-Back-N ARQ, and Selective Repeat ARQ have been extensively studied to improve packet recovery and transmission efficiency. The Stop-and-Wait protocol is simple to implement but suffers from low throughput due to idle waiting times. Go-Back-N improves efficiency by allowing multiple packets to be sent before acknowledgment, but retransmission overhead increases when packet loss occurs. Selective Repeat ARQ further enhances performance by retransmitting only lost packets, thereby improving bandwidth utilization and reducing delay. Research has also focused on congestion control and adaptive retransmission strategies to improve transport reliability. Various algorithms such as Tahoe, Reno, New Reno, and Cubic have been proposed to optimize congestion handling in transport protocols. These techniques dynamically adjust transmission rates based on network conditions to prevent congestion collapse and maintain stable communication performance. Studies indicate that adaptive congestion control significantly improves throughput and reduces packet loss in dynamic network environments.

In recent years, researchers have implemented transport protocol frameworks using programming languages such as C and C++ due to their efficiency and low-level networking capabilities. C++ provides better control over socket programming, memory management, threading, and protocol customization, making it suitable for implementing reliable communication systems. Many studies have used client-server architectures to simulate transport-layer communication, where the sender and receiver exchange packets with acknowledgment and timeout mechanisms to ensure reliable transmission. Simulation tools such as NS-3 have been widely adopted for evaluating transport protocol performance. NS-3 enables researchers to analyze network behavior under different traffic loads, packet loss conditions, and transmission delays. Performance metrics commonly studied

include throughput, end-to-end delay, packet delivery ratio, retransmission count, and network efficiency. Existing literature demonstrates that simulation-based evaluation provides valuable insights into protocol reliability and scalability.

Recent studies also emphasize the importance of customizable transport frameworks for modern applications such as IoT networks, wireless communication, and real-time systems. Traditional protocols may not perform efficiently in highly dynamic or bandwidth-constrained environments. Therefore, researchers are increasingly focusing on lightweight and adaptive reliable transport mechanisms that can be modified according to application requirements.

Based on the literature survey, it is observed that reliable transport protocols remain a critical research area in computer networking. Existing approaches provide strong reliability mechanisms, but challenges such as latency, congestion handling, retransmission overhead, and adaptability still exist. The proposed Reliable Transport Protocol Framework implemented in C++ aims to address these challenges by providing an efficient and flexible framework for reliable data communication. The project focuses on packet sequencing, acknowledgments, retransmission handling, and performance evaluation under various network conditions to improve communication reliability and overall network efficiency.

Chapter 3

Design approach and methodology

3.1 Design Approach

- Designed a reliable data transfer system using UDP protocol
- Implemented using C++ socket programming
- Developed two main modules:
 - o Sender
 - o Receiver

Sender Side:

- Generates packets with sequence numbers
- Sends data using UDP (sendto())
- Waits for acknowledgement
- Uses timeout mechanism (select())
- Retransmits packet if ACK is not received

Receiver side:

- Receives packets using `recvfrom()`
- Extracts sequence number and data
- Sends acknowledgement back to sender
- Simulates ACK loss for testing retransmission

Key Techniques Used:

- Stop-and-Wait ARQ protocol
- Sequence number-based transmission
- Timeout handling
- Retransmission mechanism
- Packet loss simulation

3.2 Code for Simulation

UDP_Sender.cpp

```
#include <iostream>
#include <cstring>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/select.h>

using namespace std;

struct Packet {
    int seqNo;
    char data[1024];
};

int main() {
    int sockfd;
    struct sockaddr_in serverAddr, recvAddr;
    socklen_t addrLen = sizeof(recvAddr);

    Packet pkt;
    int ack;

    string messages[] = {"Hello", "Packet 2", "Reliable", "END"};

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("socket");
        return 1;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));
    memset(&recvAddr, 0, sizeof(recvAddr));

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(9093);

    inet_pton(AF_INET, "192.168.0.100", &serverAddr.sin_addr);

    cout << "Sender started...\n";

    for (int i = 0; i < 4; i++) {
        pkt.seqNo = i;
        strcpy(pkt.data, messages[i].c_str());

        bool ackReceived = false;

        while (!ackReceived) {
            sendto(sockfd, &pkt, sizeof(pkt), 0,
                (struct sockaddr*)&serverAddr, sizeof(serverAddr));

            cout << "\nSent Packet " << pkt.seqNo << endl;

            fd_set readfds;
            FD_ZERO(&readfds);
            FD_SET(sockfd, &readfds);

            struct timeval tv;
            tv.tv_sec = 2;
            tv.tv_usec = 0;
```

```

        int rv = select(sockfd + 1, &readfds, NULL, NULL, &tv);

        if (rv > 0) {
            int n = recvfrom(sockfd, &ack, sizeof(ack), 0,
                             (struct sockaddr*)&recvAddr, &addrLen);

            if (n > 0 && ack == pkt.seqNo) {
                cout << "ACK received: " << ack << endl;
                ackReceived = true;
            } else {
                cout << "Wrong ACK\n";
            }
        } else {
            cout << "Timeout -> Resending Packet " << pkt.seqNo << endl;
        }
    }
}

close(sockfd);
return 0;
}

```

UDP_receiver.cpp

```

#include <iostream>
#include <cstring>
#include <arpa/inet.h>
#include <unistd.h>

using namespace std;

struct Packet {
    int seqNo;
    char data[1024];
};

int main() {
    int sockfd;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrLen = sizeof(clientAddr);

    Packet pkt;
    int ack;

    // Create socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("socket");
        return 1;
    }

    memset(&serverAddr, 0, sizeof(serverAddr));
    memset(&clientAddr, 0, sizeof(clientAddr));

    serverAddr.sin_family = AF_INET;

```

```

serverAddr.sin_addr.s_addr = inet_addr("192.168.0.100");

serverAddr.sin_port = htons(9093);

// Bind
if (bind(sockfd, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) < 0) {
    perror("bind");
    return 1;
}

cout << "Receiver running on 192.168.0.100:9093...\n";

while (true) {
    memset(&pkt, 0, sizeof(pkt));

    int n = recvfrom(sockfd, &pkt, sizeof(pkt), 0,
                    (struct sockaddr*)&clientAddr, &addrLen);

    if (n > 0) {
        cout << "\nPacket received:\n";
        cout << "Seq: " << pkt.seqNo << endl;
        cout << "Data: " << pkt.data << endl;

        ack = pkt.seqNo;

        static bool ackLost = false;
        if (pkt.seqNo == 1 && !ackLost) {
            cout << "Simulating ACK loss...\n";
            ackLost = true;
            continue;
        }
    }
}

```

```

        sendto(sockfd, &ack, sizeof(ack), 0,
                (struct sockaddr*)&clientAddr, addrLen);

        cout << "ACK " << ack << " sent\n";

        if (strcmp(pkt.data, "END") == 0) {
            cout << "Done.\n";
            break;
        }
    } else {
        perror("recvfrom");
    }
}

close(sockfd);
return 0;
}

```

3.3 Code for MATLAB

```
clc;
clear;
close all;

figure;
axis([0 10 0 5]);
hold on;

totalPackets = 4;
i = 1;
retransmissions = 0;

while i <= totalPackets
    success = rand > 0.3;

    for x = 1:0.2:7
        cla;
        text(1, 2.5, 'SENDER', 'FontSize', 12);
        text(8, 2.5, 'RECEIVER', 'FontSize', 12);

        plot(x, 2.5, 'bo', 'MarkerSize', 10, 'MarkerFaceColor', 'b');
        text(x, 3, ['Pkt ' num2str(i)]);

        text(1, 4.5, ['Retransmissions: ' num2str(retransmissions)]);
        pause(0.05);
    end

    if success

        for x = 7:-0.2:1
            cla;
            text(1, 2.5, 'SENDER');
            text(8, 2.5, 'RECEIVER');

            plot(x, 2.5, 'go', 'MarkerSize', 10, 'MarkerFaceColor', 'g');
            text(x, 3, ['ACK ' num2str(i)]);

            text(1, 4.5, ['Retransmissions: ' num2str(retransmissions)]);
            pause(0.05);
        end
    end
end
```

MATLAB Online

```

        i = i + 1;

    else
        retransmissions = retransmissions + 1;
        text(3.5, 4, 'Timeout! Retransmitting...', 'Color', 'r');
        pause(1);
    end
end

cla;
text(3, 3, 'Transmission Complete!', 'FontSize', 14);
text(3, 2.5, ['Total Packets: ' num2str(totalPackets)]);
text(3, 2, ['Retransmissions: ' num2str(retransmissions)]);

PDR = (totalPackets / (totalPackets + retransmissions)) * 100;
text(3, 1.5, ['PDR: ' num2str(PDR) ' %']);

```


Chapter 4

Implementation Details

Tools and Technologies Used

- C++ Programming Language
- Ubuntu / Linux Environment
- UDP Socket Programming
- GCC/G++ Compiler

Sender Module

- Generates packets with sequence numbers
- Sends packets using UDP
- Waits for acknowledgement
- Retransmits packet on timeout

Receiver Module

- Receives packets from sender
- Processes sequence numbers
- Sends ACK to sender
- Simulates packet loss condition

Timeout Mechanism

- Implemented using `select()` system call
- Waits for ACK for a fixed duration
- Retransmits packet if ACK is not received

Error Handling

- Packet loss handled using retransmission
- Incorrect ACK ignored
- Timeout used for recovery

Chapter 5

Results and Analysis

The project was successfully implemented and tested in a local Linux environment. Reliable communication was achieved over UDP using Stop-and-Wait ARQ protocol. The sender retransmitted packets whenever acknowledgements were not received within the timeout period.

5.1 Packet Transmission using UDP

- The system was successfully tested in a local environment
- Reliable transmission was achieved using UDP
- Packet retransmission occurred when ACK was not received
- ACK loss simulation successfully triggered retransmission

```
navyashree_sj@Navyaslaptop:~/ReliableTransportProject$ g++ udp_sender.cpp
navyashree_sj@Navyaslaptop:~/ReliableTransportProject$ ./a.out
Sender started...
```

```
Sent Packet 0
ACK received: 0

Sent Packet 1
Timeout → Resending Packet 1

Sent Packet 1
ACK received: 1

Sent Packet 2
ACK received: 2

Sent Packet 3
ACK received: 3
```

```
navyashree_sj@Navyaslaptop:~/ReliableTransportProject$ g++ udp_receiver.cpp
navyashree_sj@Navyaslaptop:~/ReliableTransportProject$ ./a.out
Receiver running on 192.168.0.100:9093...
```

```
Packet received:
Seq: 0
Data: Hello
ACK 0 sent

Packet received:
Seq: 1
Data: Packet 2
Simulating ACK loss...

Packet received:
Seq: 1
Data: Packet 2
ACK 1 sent

Packet received:
Seq: 2
Data: Reliable
ACK 2 sent

Packet received:
Seq: 3
Data: END
ACK 3 sent
Done.
```

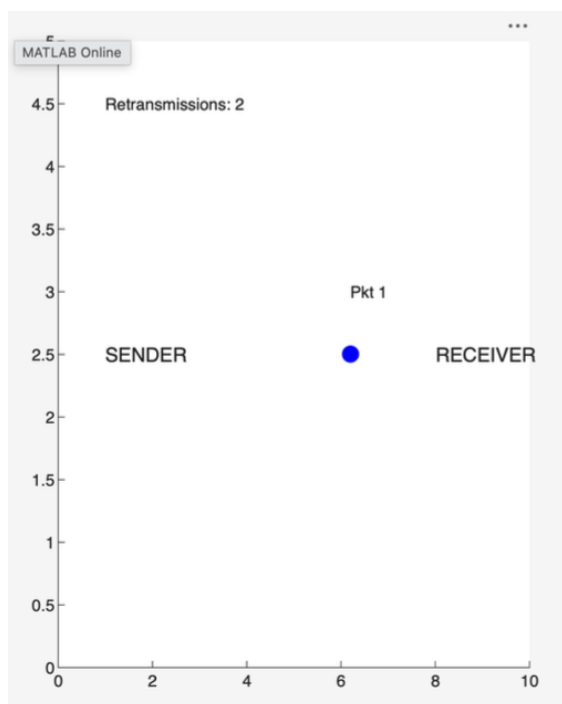
Observations:

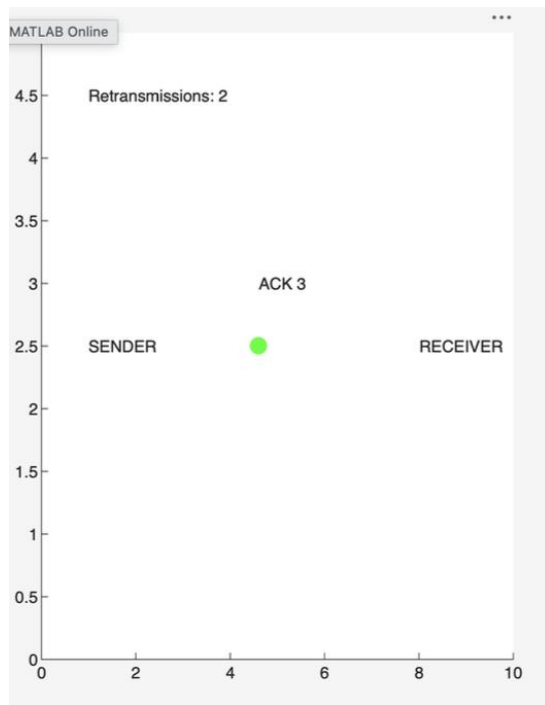
- Sender waits for ACK before sending next packet
- Timeout occurs when ACK is lost
- Retransmission ensures packet delivery

Output Behavior:

- Normal case:
 - Packet sent → ACK received
- Loss case:
 - Packet sent → Timeout → Retransmission → ACK received

5.2 Graphical Results





The figures illustrate the packet transmission and acknowledgment process between a sender and a receiver in a communication network. In the first figure, a data packet labeled “Pkt 1” is shown traveling from the sender toward the receiver, indicating the transmission of data across the network. The display also indicates that two retransmissions have occurred, suggesting that previous packet transmissions may have been lost or corrupted due to network errors or delays. In the second figure, an acknowledgment packet labeled “ACK 3” is transmitted from the receiver back to the sender, confirming the successful reception of packets. The acknowledgment mechanism helps ensure reliable communication by informing the sender that the data has been received correctly, while retransmissions improve data delivery reliability in case of packet loss.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

- Reliable data transfer successfully implemented using UDP
- Stop-and-Wait ARQ ensured packet reliability
- Retransmission mechanism handled packet loss effectively
- Project demonstrated practical networking concepts clearly
- Reliability achieved at application layer

6.2 Future Scope

- Implement Go-Back-N ARQ
- Implement Selective Repeat ARQ
- Add sliding window mechanism
- Improve efficiency and throughput
- Extend to real-time multimedia communication
- Integrate congestion control mechanisms

Bibliography

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed., Pearson, 2017.
- [2] B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013.
- [3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011.
- [4] K. Fall and K. Varadhan, *The NS Manual (formerly ns Notes and Documentation)*, The VINT Project, 2011.
- [5] M. Pathan and R. Buyya, “A Taxonomy and Survey of Content Delivery Networks,” University of Melbourne, 2007.
- [6] S. Wang, J. Bi, and J. Wu, “A Survey on Content Delivery Networks,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, 2018.
- [7] S. Q. Zhuang, B. Y. Zhao, A. D. Joseph, R. H. Katz, and J. D. Kubiatowicz, “Bayeux: An Architecture for Scalable and Fault-tolerant Wide-area Data Dissemination,” *IEEE Network*, vol. 15, no. 1, 2001.
- [8] V. Jacobson, “Congestion Avoidance and Control,” *ACM SIGCOMM*, 1988.
- [9] G. Pallis and A. Vakali, “Insight and Perspectives for Content Delivery Networks,” *Communications of the ACM*, vol. 49, no. 1, 2006.
- [10] T. V. Lakshman and U. Madhow, “The Performance of TCP/IP for Networks with High Bandwidth-delay Products,” *IEEE/ACM Transactions on Networking*, vol. 5, no. 3, 1997.